eBPF 开发的10个实战陷阱

黄竹刚

陕西 · 西安　2026年4月18日

一：bpf_get_current_pid_tgid()

一：bpf_get_current_pid_tgid()

- 返回的是一个 64 位整型
- 高 32 位是 TGID（用户态理解的进程 PID）
- 低 32 位是 PID（用户态理解的线程 TID）

一：bpf_get_current_pid_tgid()

- 返回的是一个 64 位整型
- 高 32 位是 TGID（用户态理解的进程 PID）
- 低 32 位是 PID（用户态理解的线程 TID）

- pid = bpf_get_current_pid_tgid() >> 32;
- tid = bpf_get_current_pid_tgid();

https://docs.ebpf.io/linux/helper-function/bpf_get_current_pid_tgid/

二： bpf_get_current_comm()

二： bpf_get_current_comm()

- 最长 16 字节（TASK_COMM_LEN）
- 超过会被截断
- 获取的是线程名称，并一定是进程名称

二： bpf_get_current_comm()

- 最长 16 字节（TASK_COMM_LEN）
- 超过会被截断
- 获取的是线程名称，并一定是进程名称

- task->mm->arg_start
- task->mm->arg_end

https://docs.ebpf.io/linux/helper-function/bpf_get_current_comm/

三：bpf_get_current_task ()

三：bpf_get_current_task ()

- 可能是一个内核线程（e.g. ksoftirqd/1）
- 可能是被中断抢占的其他用户进程

三：bpf_get_current_task ()

- 可能是一个内核线程（e.g. ksoftirqd/1）
- 可能是被中断抢占的其他用户进程

- task->flags & PF_KTHREAD
- bpf_in_interrupt()
- bpf_in_task()
- bpf_in_serving_softirq()
- bpf_in_hardirq()

https://docs.ebpf.io/linux/helper-function/bpf_get_current_task/
https://github.com/torvalds/linux/blob/1f5ffc672165ff851063a5fd044b727ab2517ae3/tools/testing/selftests/bpf/bpf_experimental.h#L519

四：syscalls/sys_enter_execve

四：syscalls/sys_enter_execve

- 无法获取定义为常量字符串的程序名称
- bpf_probe_read_user_str()
- page fault

四：syscalls/sys_enter_execve

- 无法获取定义为常量字符串的程序名称
- bpf_probe_read_user_str()
- page fault

- 换 hook 点
- 比如，sched/sched_process_exec

https://docs.ebpf.io/linux/helper-function/bpf_probe_read_user_str/
https://lists.iovisor.org/g/iovisor-dev/topic/accessing_user_memory_and/21386221
https://mozillazg.com/2024/03/ebpf-tracepoint-syscalls-sys-enter-execve-can-not-get-filename-argv-values-case.html

五：kretprobe

五：kretprobe

- maxactive
- 限制最大并发实例数
- 超出限制时待执行的 eBPF 程序将被跳过

五：kretprobe

- maxactive
- 限制最大并发实例数
- 超出限制时待执行的 eBPF 程序将被跳过
- 使用 fexit 代替
- 指定 maxactive 参数

https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_KPROBE/
https://docs.kernel.org/trace/kprobes.html

六：IP/TCP Options

六：IP/TCP Options

- 解析时硬编码 Header 长度导致结果异常
- 存在可变长度的 Options

六：IP/TCP Options

- 解析时硬编码 Header 长度导致结果异常
- 存在可变长度的 Options

- (iph->ihl) * 4
- (tcph->doff) * 4

七：TC_ACT_OK

七：TC_ACT_OK

- 其他 TC 程序可能会被跳过

七：TC_ACT_OK

- 其他 TC 程序可能会被跳过
- TC_ACT_UNSPEC

https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_SCHED_CLS/

八：TC qdisc

八：TC qdisc

- 硬编码 priority 和 handle 导致冲突
- 无脑删除或替换已有 qdisc
- 误删其他 TC 程序

八：TC qdisc

- 硬编码 priority 和 handle 导致冲突
- 无脑删除或替换已有 qdisc
- 误删其他 TC 程序

- 使用 TCX
- 循环探测可用的 handle
- 只删除自己创建的 qdisc

https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_SCHED_CLS/

九：percpu map

九：percpu map

- 运行中的 eBPF 程序被抢占
- 数据损坏

九：percpu map

- 运行中的 eBPF 程序被抢占
- 数据损坏

- native xdp / tc bpf 使用安全
- ringbuf
- bpf_preempt_disable

https://t.me/c/1319082550/6571
https://lore.kernel.org/bpf/CAMy7=ZWPc279vnKK6L1fssp5h7cb6cqS9_EuMNbfVBg_ixmTrQ@mail.gmail.com/T/

十：lru map

十：lru map

- 小规模 (size < 128) 无法 LRU
- 丢数据
- 死锁
- 内核 BUG

十：lru map

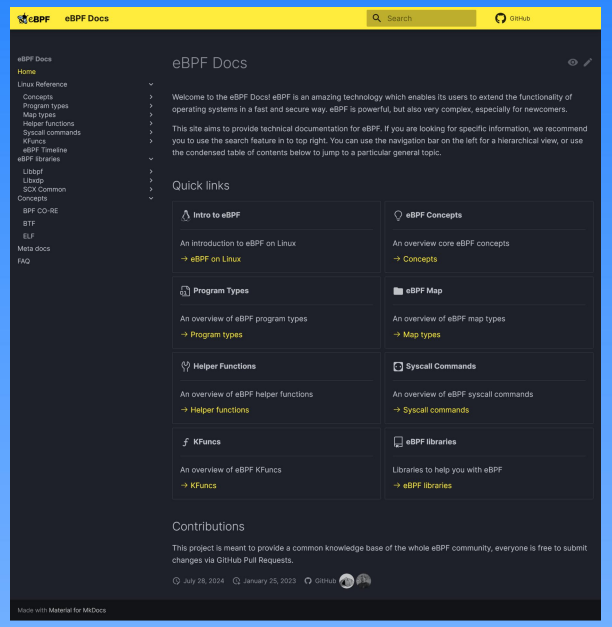
- 小规模 (size < 128) 无法 LRU
- 丢数据
- 死锁
- 内核 BUG

- 能不用就不用
- > 4k, cpu affinity（tc/xdp）

https://t.me/c/1319082550/5318
https://blog.leonhw.com/post/ebpf-talk-157-lru-hash-map-issues/

资源推荐

docs.ebpf.io



Slack#ebpf

资源推荐

Telegram 频道

@THEB1ACKPARADE

t.me/TheB1ackParade

擅长解决各种疑难杂症，能解常人所不能解的 BUG。这个频道时常会分享各种 eBPF 和 DEBUG 技巧以及新颖的开创性想法。

@EBPFTALK001

t.me/eBPFTalk001

在持续高强度给 eBPF 社区做贡献，不是在给内核提交 eBPF 补丁，就是在开发既实用又强大的 eBPF 应用。这个频道时常会分享各种 eBPF 知识以及新项目、新贡献。

谢谢！